

Tarea # 4

En el código fuente del Kernel de Linux (específicamente en el archivo include/net/sock.h), la función que decide si rechazar o aceptar una nueva conexión se llama `sk_acceptq_is_full()`.

La condición matemática que usa el kernel **no es** "mayor o igual a", sino estrictamente "mayor que" ($>$).

```
// Código fuente real del Kernel de Linux
static inline bool sk_acceptq_is_full(const struct sock *sk)
{
    return sk_acceptq_len(sk) > sk->sk_max_ack_backlog;
}
```

Aplicando esto en un laboratorio donde backlog = 1, tenemos:

- El Cliente 1 entra "directo" y es atendido por el servidor, tamaño de la cola es 0.
- Entra el Cliente 2: El kernel pregunta: ¿La cantidad actual en cola (0) es estrictamente mayor que tu límite (1)? $0 > 1$ es Falso, por lo que lo deja pasar al backlog. Estado de la cola: Hay 1 cliente esperando.
- Entra el Cliente 3: El kernel pregunta: ¿La cantidad actual en cola (1) es estrictamente mayor que tu límite (1)? $1 > 1$ es Falso. ¡También lo deja pasar al backlog! Y ahora, la cola ya tiene 2 conexiones en espera.
- Entra el Cliente 4: El kernel pregunta: ¿La cantidad actual en cola (2) es estrictamente mayor que tu límite (1)? $2 > 1$ es Verdadero. ¡Rechazado! (Connection refused / Timeout).

Vamos a realizar un pequeño laboratorio para comprobarlo.

Las capturas de pantalla se realizaron del codespaces de Github, pero es indistinto si lo hacen en su Linux local.

En la terminal 1 se cambian a usuario root y prueban:

```
cat /proc/sys/net/ipv4/tcp_syncookies
```

```
cat /proc/sys/net/ipv4/tcp_syn_retries
```

deben tener valores 0 y 1 respectivamente. En esta captura de pantalla tienen valores 1 y 6 respectivamente por lo que estará reintentando hasta por 180 segundos lo cual invalidaría nuestro laboratorio. Para que nuestro laboratorio funcione deben cambiarlos con:

```
sudo sysctl -w net.ipv4.tcp_syncookies=0
```

```
sudo sysctl -w net.ipv4.tcp_syn_retries=1
```

Después de cambiar los valores, pueden comprobarlos repitiendo el comando cat:

[Vista previa] README.md X

Arquitectura Cliente Servidor

Bienvenidas y bienvenidos al repositorio oficial de la asignatura. Aquí encontrarán el código fuente de los progr

Para consultar los temarios, apuntes en PDF y la bibliografía, visita el [Sitio Web de la Asignatura](#).

Problemas Salida Consola de depuración Terminal Puertos

```
@CRomanZamitiz →/workspaces/arquitectura-cliente-servidor (main) $ sudo su
root →/workspaces/arquitectura-cliente-servidor (main) $ cat /proc/sys/net/ipv4/tcp_syncookies
1
root →/workspaces/arquitectura-cliente-servidor (main) $ cat /proc/sys/net/ipv4/tcp_syn_retries
6
root →/workspaces/arquitectura-cliente-servidor (main) $ sudo sysctl -w net.ipv4.tcp_syncookies=0
net.ipv4.tcp_syncookies = 0
root →/workspaces/arquitectura-cliente-servidor (main) $ sudo sysctl -w net.ipv4.tcp_syn_retries=1
net.ipv4.tcp_syn_retries = 1
root →/workspaces/arquitectura-cliente-servidor (main) $ cat /proc/sys/net/ipv4/tcp_syncookies
0
root →/workspaces/arquitectura-cliente-servidor (main) $ cat /proc/sys/net/ipv4/tcp_syn_retries
1
root →/workspaces/arquitectura-cliente-servidor (main) $
```

En esta misma ventana lanzan el servidor del programa ejemplo_listen:

```
root →/workspaces/arquitectura-cliente-servidor (main) $ cd tema3
root →/workspaces/arquitectura-cliente-servidor/tema3 (main) $ cc ejemplo_listen.c -o ejemplo_listen
root →/workspaces/arquitectura-cliente-servidor/tema3 (main) $ ./ejemplo_listen 4898
Servidor iniciado en el puerto 4898.

[!] Servidor en estado LISTEN. Esperando en accept()...
```

En otra terminal que simula estar del lado del servidor, lanzan el comando:

`watch -n 1 "ss -atn 'sport = :4898'"`

[Vista previa] README.md X

Arquitectura Cliente Servidor

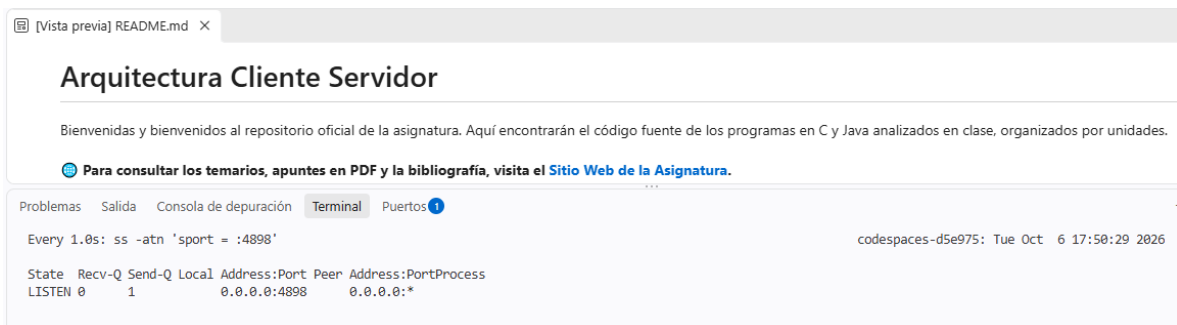
Bienvenidas y bienvenidos al repositorio oficial de la asignatura. Aquí encontrarán el código fuente de los programas en

Para consultar los temarios, apuntes en PDF y la bibliografía, visita el [Sitio Web de la Asignatura](#).

Problemas Salida Consola de depuración Terminal Puertos 1

```
@CRomanZamitiz →/workspaces/arquitectura-cliente-servidor (main) $ watch -n 1 "ss -atn 'sport = :4898'"
```

Al dar enter, observarán una salida similar a la siguiente:



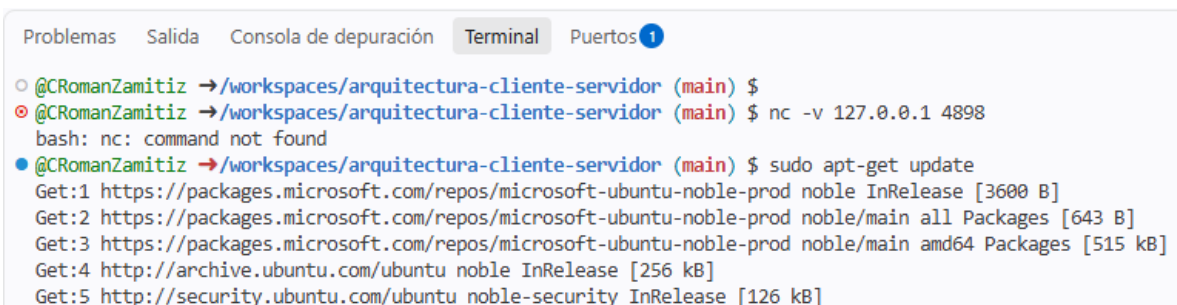
The screenshot shows a web browser window with a tab titled '[Vista previa] README.md'. The page content includes the title 'Arquitectura Cliente Servidor', a welcome message, and a link to the assignment website. Below the browser window, a terminal window is open, showing a netcat listener on port 4898. The terminal output includes the command 'ss -atn 'sport = :4898'', the status 'LISTEN 0', and the peer address '0.0.0.0:*'.

En otra terminal que simula ser el cliente 1, lanzamos el comando

```
nc -v 127.0.0.1 4898
```

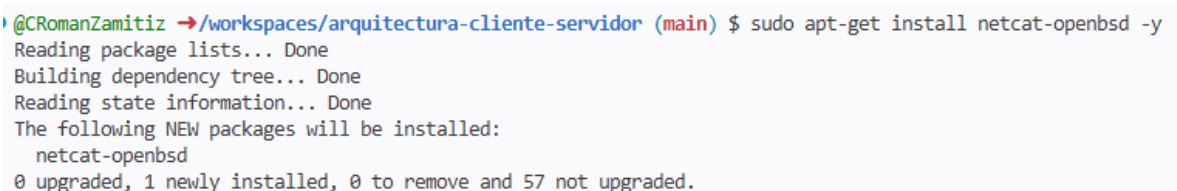
Si les indica que no existe el comando pueden descargar la herramienta nc

La siguiente captura de pantalla es para descargarla desde Ubuntu, que es el sistema del codespaces de Github:



The screenshot shows a terminal window with the following commands and output:

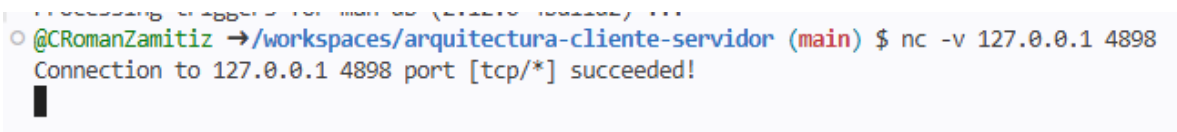
```
@CRomanZamitiz →/workspaces/arquitectura-cliente-servidor (main) $  
@CRomanZamitiz →/workspaces/arquitectura-cliente-servidor (main) $ nc -v 127.0.0.1 4898  
bash: nc: command not found  
@CRomanZamitiz →/workspaces/arquitectura-cliente-servidor (main) $ sudo apt-get update  
Get:1 https://packages.microsoft.com/repos/microsoft-ubuntu-noble-prod noble InRelease [3600 B]  
Get:2 https://packages.microsoft.com/repos/microsoft-ubuntu-noble-prod noble/main all Packages [643 B]  
Get:3 https://packages.microsoft.com/repos/microsoft-ubuntu-noble-prod noble/main amd64 Packages [515 kB]  
Get:4 http://archive.ubuntu.com/ubuntu noble InRelease [256 kB]  
Get:5 http://security.ubuntu.com/ubuntu noble-security InRelease [126 kB]
```



The screenshot shows a terminal window with the following commands and output:

```
@CRomanZamitiz →/workspaces/arquitectura-cliente-servidor (main) $ sudo apt-get install netcat-openbsd -y  
Reading package lists... Done  
Building dependency tree... Done  
Reading state information... Done  
The following NEW packages will be installed:  
  netcat-openbsd  
0 upgraded, 1 newly installed, 0 to remove and 57 not upgraded.
```

Si ya lo tienen instalado, lanzan el comando:



The screenshot shows a terminal window with the following command and output:

```
@CRomanZamitiz →/workspaces/arquitectura-cliente-servidor (main) $ nc -v 127.0.0.1 4898  
Connection to 127.0.0.1 4898 port [tcp/*] succeeded!
```

Observen que el cliente 1 indica “succeeded” que indica que la aceptación fue exitosa.

Cuando lanzan el cliente 2 y cliente 3, el resultado es similar a cliente 1:



The screenshot shows the VS Code interface with a terminal window. The terminal output shows a successful connection to 127.0.0.1 port 4898. The terminal title bar indicates 'Terminal' and 'Puertos 1'. The terminal content shows the command `nc -v 127.0.0.1 4898` and the output `Connection to 127.0.0.1 4898 port [tcp/*] succeeded!`. The terminal also shows the command `nc` being executed, and the output `nc` is displayed. The terminal also shows the command `nc` being executed, and the output `nc` is displayed.



The screenshot shows the VS Code interface with a terminal window. The terminal output shows a successful connection to 127.0.0.1 port 4898. The terminal title bar indicates 'Terminal' and 'Puertos 1'. The terminal content shows the command `nc -v 127.0.0.1 4898` and the output `Connection to 127.0.0.1 4898 port [tcp/*] succeeded!`. The terminal also shows the command `nc` being executed, and the output `nc` is displayed. The terminal also shows the command `nc` being executed, and the output `nc` is displayed.

Cuando lanzan el cliente 4, éste es rechazado:



The screenshot shows the VS Code interface with a terminal window. The terminal output shows a failed connection to 127.0.0.1 port 4898. The terminal title bar indicates 'Terminal' and 'Puertos 1'. The terminal content shows the command `nc -v 127.0.0.1 4898` and the output `nc: connect to 127.0.0.1 port 4898 (tcp) failed: Connection timed out`. The terminal also shows the command `nc` being executed, and the output `nc` is displayed. The terminal also shows the command `nc` being executed, and the output `nc` is displayed.

En este punto, regresen a la terminal del comando ss y observen la salida:

[Vista previa] README.md

Arquitectura Cliente Servidor

Bienvenidas y bienvenidos al repositorio oficial de la asignatura. Aquí encontrarán el código fuente de los programas en C y Java analizados en clase.

Para consultar los temarios, apuntes en PDF y la bibliografía, visita el [Sitio Web de la Asignatura](#).

Problemas
Salida
Consola de depuración
Terminal
Puertos

Every 1.0s: ss -atn 'sport = :4898'
codespaces-d5e975: Tue Oct 6 18:49:09 2026

sudo
watch
nc
nc
nc
nc
bash

State	Recv-Q	Send-Q	Local Address:Port	Peer Address:Port	Process
LISTEN	2	1	0.0.0.0:4898	0.0.0.0:*	
ESTAB	0	0	127.0.0.1:4898	127.0.0.1:52656	
ESTAB	0	0	127.0.0.1:4898	127.0.0.1:52886	
ESTAB	0	0	127.0.0.1:4898	127.0.0.1:48496	

Analicemos paso a paso lo que está ocurriendo a nivel de red y del kernel de Linux con los valores observados en las terminales:

1. Parámetro del backlog (Send-Q):

En la salida del comando ss para el socket en estado LISTEN, la columna Send-Q marca el valor 1. Esto refleja directamente el valor asignado al segundo argumento de la función en C: listen(sockfd, 1).

2. Conexiones completadas y encoladas (ESTAB y Recv-Q):

- El primer cliente (nc) completó el saludo de tres vías (3-way handshake) y fue extraído de la cola al ser atendido por la llamada accept() del servidor.
- Los clientes 2 y 3 completaron también su enlace TCP con éxito (Connection to ... succeeded!). En Linux, la cola de conexiones establecidas pendientes de ser aceptadas admite hasta el límite evaluado por el kernel (backlog + 1), por lo que ambas quedaron encoladas a la espera. Esto se comprueba en la columna Recv-Q, que muestra un valor de 2.
- La suma de la conexión atendida más las dos retenidas en la cola da como resultado las 3 conexiones en estado ESTAB que lista ss.

3. Saturación y descarte en el Cliente 4:

- Al alcanzar la capacidad máxima de la cola, el kernel ya no puede almacenar más conexiones establecidas.
- Debido a que en el paso previo se desactivaron los syncookies (tcp_syncookies = 0) y se limitaron las retransmisiones (tcp_syn_retries = 1), el sistema descarta los paquetes SYN entrantes del cuarto cliente en lugar de responder o negociar cookies de sincronización.
- Al enviar su solicitud, no recibir respuesta del servidor y agotar de inmediato su único reintento permitido, el cuarto cliente finaliza con el mensaje:
nc: connect to 127.0.0.1 port 4898 (tcp) failed: Connection timed out.

Al cumplirse exactamente la retención de las tres conexiones en ESTAB y el descarte por tiempo límite en el intento posterior, la premisa de que deben registrarse 3 clientes con "succeeded" y el cuarto con "time out" es completamente correcta. La prueba del comportamiento del backlog fue exitosa.